

Use the Citation Checker in EduGenAI

Register this tool as an EduGenAI Extension so the assistant can verify a paper's references against OpenAlex — inside your own EduGenAI chat, with no separate LLM key.

How it works

An EduGenAI Extension is a function-calling tool: you give EduGenAI an HTTP endpoint and a function definition, and its assistant calls that endpoint as a JSON request whenever it needs to. The work splits across two sides:

- **EduGenAI's assistant** reads the paper and extracts the reference list plus the sentences that cite each source.
- **This extension endpoint** looks each reference up in OpenAlex and checks title, authors, year, journal, DOI, volume, issue and pages — deterministically, with no LLM — and returns the abstract.
- **EduGenAI's assistant** compares the citing sentence to that abstract and flags misquotes.

Because the reasoning stays on EduGenAI's side, the extension needs no LLM API key. It only wraps OpenAlex, which is free. An optional OpenAlex Premium key can be stored securely in the Headers / Azure Key Vault section (Step 3).

Before you start

Deploy this app at a public HTTPS URL — EduGenAI calls it server-to-server. Everywhere below, replace `https://YOUR-DEPLOYMENT-HOST` with your deployment's address.

Step 1 — Open the Extension builder

In EduGenAI, create a new Extension (Name, Short description, Headers, Functions).

Step 2 — Name and describe it

Name

OpenAlex Citation Verifier

Short description

Verifies a paper's references against OpenAlex: checks that each work exists and that the printed title, authors, year, journal, DOI and pages match, and returns the abstract.

Detail description (paste verbatim — this is the assistant's instruction)

Use this extension to fact-check the reference list of an uploaded paper.

Workflow:

1. Read the document and extract every entry in the reference list. For each, pull out: title, the full list of author names, year, DOI (only if printed), journal/venue, volume, issue, and page range. Also find the sentence(s) in the body where that source is cited, with one sentence of context on each side.
2. Call the "verify_references" function with all references at once.
3. For every reference the function returns:
 - If status is "not_found", flag it as a POTENTIAL HALLUCINATION.
 - If status is "fuzzy", tell the user the closest candidate(s) it returned.
 - If "field_check" contains any field with status "mismatch", report exactly which printed details are wrong (especially authors and year), showing the reference value vs the OpenAlex value.
 - Using the returned "abstract", judge whether the student's citing sentence matches what the paper is actually about. If the paper is on a different topic than the student implies, flag it as a MISQUOTE and explain in one sentence.

4. Present the results as a table sorted worst-first: hallucinations, then wrong authors/DOI/title, then misquotes, then wrong year, then minor field mismatches, then clean references. Never invent a DOI, author, or verdict that the function did not return.

Step 3 — Add the Header

In the Headers section add: **Key** Content-Type **Value** application/json.

Optional — OpenAlex Premium: add a second header X-OpenAlex-Key with your key as the value, using the Secure header values option so it is stored in Azure Key Vault.

Step 4 — Add the function

Method & URL

POST https://YOUR-DEPLOYMENT-HOST/api/verify_batch

Function definition

```
{
  "name": "verify_references",
  "description": "Verify a list of bibliographic references against OpenAlex.
  For each reference, returns whether the work exists (found / fuzzy /
  not_found), a field-by-field comparison of the printed metadata (title,
  authors, year, journal, DOI, volume, issue, pages) against the real record,
  and the abstract of the matched work.",
  "parameters": {
    "type": "object",
    "properties": {
      "references": {
        "type": "array",
        "description": "Every reference in the paper's bibliography.",
        "items": {
          "type": "object",
          "properties": {
            "title": { "type": "string" },
            "authors": { "type": "array", "items": { "type": "string" },
              "description": "Every author name printed, in order." },
            "et_al": { "type": "boolean" },
            "year": { "type": "integer" },
            "doi": { "type": "string" },
            "journal": { "type": "string" },
            "volume": { "type": "string" },
            "issue": { "type": "string" },
            "pages": { "type": "string" }
          },
          "required": ["title"]
        }
      }
    },
    "required": ["references"]
  }
}
```

Step 5 — Submit and test

Click Submit. In a chat, upload a paper and ask: “Check the citations in this paper against OpenAlex.”

What the endpoint returns

Request EduGenAI sends to `/api/verify_batch`:

```
{
  "references": [
    {
      "title": "Highly accurate protein structure prediction with AlphaFold",
      "authors": ["Smith, J.", "Jones, B."],
      "et_al": false,
      "year": 2019,
      "journal": "Science",
      "pages": "100-110"
    }
  ]
}
```

Response (abbreviated) — wrong authors, year and journal are caught, and the abstract is returned for the misquote step:

```
{
  "count": 1,
  "results": [{
    "index": 1,
    "status": "found",
    "work": {
      "title": "Highly accurate protein structure prediction with AlphaFold",
      "authors": ["John Jumper", "Richard Evans", "..."],
      "year": 2021, "venue": "Nature",
      "doi": "10.1038/s41586-021-03819-2",
      "abstract": "Proteins are essential to life ... (used for the misquote check)"
    },
    "field_mismatch_count": 3,
    "field_check": [
      { "field": "title", "status": "match" },
      { "field": "authors", "status": "mismatch",
        "reference_value": "Smith, J., Jones, B.",
        "openalex_value": "John Jumper, Richard Evans, ..." },
      { "field": "year", "status": "mismatch",
        "reference_value": 2019, "openalex_value": 2021 },
      { "field": "journal", "status": "mismatch",
        "reference_value": "Science", "openalex_value": "Nature" }
    ]
  }]
}
```

A single-reference endpoint is also available at `POST /api/verify` (same fields, no outer “references” array). Prefer the batch endpoint for a whole bibliography — one round trip, kinder to OpenAlex rate limits.

Notes & limits

- No LLM key is stored or used by the extension; OpenAlex is free and needs no key.
- OpenAlex's canonical year can differ from a printed year (online-first vs issue year) — treat a lone year mismatch as a prompt to double-check, not a verdict.
- The misquote judgement uses the abstract only, so a claim supported by the full text but not the abstract may read as uncertain. Treat results as leads for a human reviewer.
- Batch requests are capped at 200 references.